

OOP's

* Object oriented programming (OOP) :-
 OOP is a method of structuring program by building related properties and behavior into individual object.

OOP is programming paradigm that uses object and classes in programming. it aims to implement real-world entities like is

* The main concept of OOPs is to bind the data and the functions that work on that together as a single unit so that no other part of the code can access this data.

* Concept / pillar of OOPs :-

- ① class
- ② object
- ③ Inheritance
- ④ polymorphism
- ⑤ Abstraction
- ⑥ Incapsulation

① class :- A class is collection of object. A class contains the blueprints or the prototype from which the objects are being created. It is a logical entity that contains some attributes and method.

- classes are created by keyword class.
- Attributes are always public and can be accessed using the dot (.) operator eg myclass.myattribute.
- Attributes are the variables that belongs to a class.

syntax: class className:

statement

Ex: class dog:

pass

Object :- The object is an entity that has a state and behavior associated with it. It may be any real-world object like mouse, keyboard, chair.

- The identity can be considered as the name of the dog.
- State or Attri^b can be considered as the breed, age or colour of dog.
- The behavior can be considered as the whether the dog is eating or sleeping.

Syntax :- obj = Dog().

* self attribute :- self represents the instance of the class. By using the 'self' we can access the attribute and method of the class in python. It binds the attributes with the given arguments.

* Constructor :- Constructors are generally used for initializing an object. The task of constructor is to initialize (assign value) to the data members of the class when an object of the class is created. In python the `__init__()` method is called the constructor and is always called when an object is created.

Syntax : `def __init__(self):`
body of the constructor.

ex:- class student :

```
def __init__(self):  
    self.name = "pranod"  
    self.age = 23  
    self.per = 81.32  
    print("Init method")  
def show(self):  
    print("name =", self.name)  
    print("Age =", self.age)  
    print("percentage =", self.per)
```

S1 = student()

S1.show()

* Create a class company name, profit, & display the data for two object.

```
→ class company:
    def __init__(self, cn, pr):
        self.cn = cn
        self.pr = pr
    def show(self):
        print("company", self.cn)
        print("profit", self.pr)
s1 = company("ITP", 4.5)
s1.show()
s2 = company("wipro", 8.3)
s2.show()
```

* Types of class variable in python :-

- (1) Instance variable (object level variables)
- (2) Static variable (class level variable)
- (3) Local variable

* Instance variable in python :- if value of a variable changing from object to object then such variables called instance variable.

```
Ex. class student:
    def __init__(self, name, id):
        self.name = name
        self.id = id
```

```
s1 = student('Nitya', 1)
s2 = student('Anushka', 2)
print('student 1 info :')
print('Name:', s1.name)
print('Id:', s1.id)
print('student 2 info:')
print('name:', s2.name)
print('id:', s2.id)
```


Page No. _____
Date: / /

* Static Variable :- if the value of a variable is not changing from object to object, such type of variables are called static variables or class level variable. We can access static variable either by class name or by object name. Accessing static variables with class names is highly recommended than object names.

* What is local variable in python :-
The variable which we declare inside of the method is called a local variable. generally for temporary usage we create local variables to use within the method. The scope of these variables is limited to the method in which they are declared. They are not accessible outside of the methods.

* Types of Method :-

① Instance method :- if we use instance variables inside a method such methods are called instance methods. The instance method perform a set of action on the data/value provided by the instance variable.

* write a program using a student class with instance methods to process the data for several students.

```
class student:
```

```
    def __init__(self, n="", m=0):
```

```
        self.n = n
```

```
        self.m = m
```

```
    def show(self):
```

```
        print('name =', self.n)
```

```
        print('marks =', self.m)
```

```
    def result(self):
```

```
        if (self.m > 75):
```

```
            print('distinction')
```



```

elif (self.m >= 60):
    print ("first class")
elif (self.m >= 50):
    print ("pass")
else:
    print ('fail')
n = int (input ("How many student ?"))
i = 1
while (i <= n)
    nm = input ("student name")
    mk = input ("student marks")
    st = student (nm, mk)
    st.show()
    st.result()
    i += 1
    print ("....")
    
```

* Create a class Bank where deposit, withdraw and exit are the menu to perform the bank transaction handle by using instance method.

```

→ class Bank:
    def __init__(self):
        self.balance = 0
    def deposit(self):
        amount = int (input ("Enter deposit amount:"))
        self.balance += amount
        print ("Amount deposit", amount)
    def withdraw(self):
        amount = int (input ("Enter withdraw amount:"))
        if self.balance >= amount:
            self.balance -= amount
            print ("successful withdrawn", amount)
        else:
            print ("Insufficient balance")
    
```


Page No.
 Date: / /

```

def display (self):
    print ("Net available balance = " + self.balance)
s = Bank ()
s.deposit ()
s.withdraw ()
s.display ()
  
```

* Accessor method :- Accessor method simply access or read data of variable they do not modify data in variables. Accessor methods are generally written in form of get hence also called as getter method.

* Mutator Method :- mutator methods are the methods which not only read the data but also modify them. They are written in form of set hence they are also called as setter method.

* WAP using accessor & mutator methods :-

```

→ class student :
    def setName (self, name)
        self.name = name
    def getName (self) :
        print (self.name)
    def setmarks (self, mark) :
        self.marks = mark
    def getmarks (self) :
        print (self.marks)
n = int (input ("No of object"))
i = 1
while (i <= n)
    s = student ()
    nm = input ("Enter student name : ")
    s.setName (nm)
    mk = int (input ("marks of student"))
  
```


s. setmarks(mk)
s. getname ()
s. getmarks ()
i += 1
print (".....")

* What is class method ?

A method inside a class with @classmethod decorator is called a class method. we will discuss this in next chapter. inside the class method we can declare and initialize static variables by using the class's name.

Ex. class Bird:

wings = 2

@classmethod

def fly (cls, name)

print ('{} fly with {} wings'.format
(name, cls.wings))

b. birds ()

birds.fly ('sparrow')

birds.fly ('crow')

* Static Method :-

- Before method if we write @staticmethod decorator then that method will become a static method. we will learn more about static method later.

- The static methods, in general, utility methods. inside these method we won't use any instance or class variable. No arguments like cls or self are required at the time of declaration.

- we can access static methods by using class name or object reference.

★ write a program to demonstrate static method

```
class myclass:
```

```
    n = 0
```

```
    def __init__(self):
```

```
        myclass.n = myclass.n + 1
```

```
    @ staticmethod
```

```
    def noobjects():
```

```
        print("No of object created", myclass.n)
```

```
a1 = myclass()
```

```
a2 = myclass()
```

```
a3 = myclass()
```

```
myclass.noobjects()
```

→ create a class sample create a static method that calculate square root of a given number.

```
→ import math
```

```
class sample:
```

```
    @ staticmethod
```

```
    def calculate(x):
```

```
        n = math.sqrt(x)
```

```
        return n
```

```
num = eval(input("Enter any number"))
```

```
result = samp.calculate(num)
```

```
print("square root", result)
```

★ class person:

```
    def __init__(self):
```

```
        self.name = 'Arushi'
```

```
        self.dob = self.dob()
```

```
    def show(self):
```

```
        print("person name", self.name)
```

```
class DOB:
```

```
    def __init__(self):
```

```
        self.d = 91
```

```
        self.m = 3
```



```

self.y = 2023
def display(self):
    print("{}-{}-{}".format(self.d, self.m, self.y))
p1 = person()
p1.show()
p1.dob.display

```

→ Inner class - A class defined in another class is known as inner class or nested class. If an object is created using child class means inner class then object can also be used by parent class or root class. A parent class can have one or more inner classes but generally inner classes are avoided.

We can make our code even more object-oriented by using an inner class. A single object of the class can hold multiple sub-object. We can use multiple sub-objects to give a good structure to our program.

Ex.

```

class color:
    def __init__(self):
        self.name = 'green'
        self.lg = self.lightgreen()
    def show(self):
        print('Name:', self.name)
class lightgreen:
    def __init__(self):
        self.name = 'light green'
        self.code = '024arc'
    def display(self):
        print('Name:', self.name)
        print('code:', self.code)
outer = color()
outer.show()
g = outer.lg
g.display()

```


* Four concept or pillar of oop's *

① Inheritance :- Inheritance is an important property of oop. Inheritance is the process of the inheriting the properties of the parent class into child class.

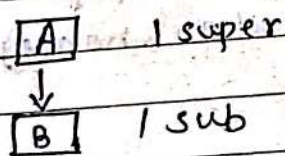
The main advantages of inheritance is it provides reusability of a code we don't have to write a code again and again.

The already existing class is called superclass or based class ~~is~~ and newly created class called as subclass or derived class.

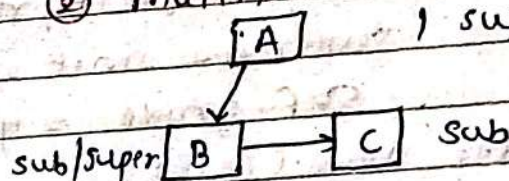
syntax:- class Subclass (superclass)

* Types of Inheritance :-

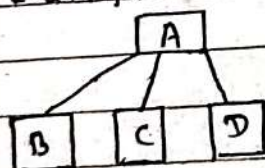
① Single inheritance



② multilevel inheritance.



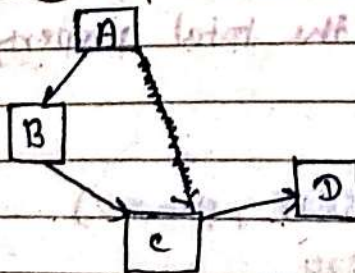
③ Hierarchical inheritance



④ Multiple inheritance.



⑤ Hybrid inheritance



Parent
Super (base)
Subclass (derived)
Trans. (inheritance)
write carefully

* Single inheritance :- Deriving one class or more subclasses from a single based class is called single inheritance.

* Create a class student having members roll number and name inherit it test class and display grade of a student for single object.

```

→ class student:
    def __init__(self, roll no, name):
        self.roll no = roll no
        self.name = name
    def show(self):
        print("name = 33 It Roll no = 33", format(
            self.name, self.roll no))
class test(student):
    def __init__(self, roll no, name, grade):
        super().__init__(roll no, name)
        self.grade = grade
    def show(self):
        super().show()
        print("student grade", self.grade)
t = test(11, 'ABC', 'first class')
t.show()

```

* Create a class father having a member properties inherit the property in class son and add them in his property display the total property of son of single object.

```

→ class father:
    def __init__(self, pro=0):
        self.pro = pro
    def show(self):
        print("father property", self.pro)
class son(father):
    def __init__(self, father's

```


Page No.
 Date. / /

```

def __init__(self, pro=0, property=1):
    super().__init__(pro)
def show(self):
    super().show()
    print('Total property = ', self.pro + self.prop)
s1 = son(1000000, 100000)
s1.show()

```

~~* Super method *~~

* Super() function :- The super() function is used to give access to method and properties of a parent or sibling class. The super() function return an object that represents the parent class.
 Syntax :- super().

* Create a program to access base class Constructor and method in the subclass using Super function where square is a base class carry area method and rectangle its subclass rectangle also having method area.

→ Class Square:

```

def __init__(self, x):
    self.x = x

```

```

def area(self):

```

```

    print("Area of square", self.x * self.x)

```

```

class rectangle(Square):

```

```

    def __init__(self, x, y):

```

```

        super().__init__(x)

```

```

        self.y = y

```

```

    def area(self):

```

```

        super().area()

```

```

        print("Area of rectangle", self.x * self.y)

```

```

l, b = [float(i) for i in input('Enter 2 no').split(" ")]

```

```

r = rectangle(l, b)

```

```

r.area()

```


* Create a class Company having members year of experience and Salary increment inherit into person class and display net salary of the person as per its year of experience.

```

class Company:
    def __init__(self, yoe, si):
        self.yoe = yoe
        self.si = si
    def show(self):
        if (self.yoe > 5):
            print("Salary increment = 10%")
        else:
            print("NO increment")
class Person(Company):
    def __init__(self, yoe, si, net):
        super().__init__(yoe, si)
        self.net = net
    def show(self):
        super().show()
        print("Net salary =", self.net)
person(5, 10, 25000)
show().
    
```

* Multiple inheritance :- Deriving subclasses from multiple base classes is called as multiple inheritance in this type of inheritance more than one superclass and there may be one or more subclasses.

* Problem in multiple inheritance :-

problem in multiple inheritance if subclass has constructor override the superclass constructor hence the superclass constructor is not available to the subclass but creating constructor is very common to initialize the instance variable.

Page No. _____
Date: / /

* Method Resolution Order (MRO) :- In multiple inheritance scenario any specified attribute or method is such first in the current class, is not found in search continue into parent class in depth first left to right fashion without searching the same class twice searching in this way is called method resolution order (MRO)

* There are three principle of MRO :-

- ① to search for subclass before going to its base classes.
- ② when a class is inherited from the classes, it search in the order from the left to right in the base class.
- ③ it will not visit any class more than ones that means a class in the inheritance hirachi is travel only ones.

→ write a program to demonstrate MRO using multiple inheritance

```
class A(object):  
    def method(self):  
        print('A class method')  
        super.method()
```

```
class B(object):  
    def method(self):  
        print('B class method')  
        super.method()
```

```
class C(object):  
    def method(self):  
        print('C class method')  
        super.method()
```

```
class X(A, B):  
    def method(self):  
        print('X class method')  
        super.method()
```

```
class Y(B, C):  
    def method(self):  
        print('Y class method')  
        super.method()
```



```
class P(x, y, z):
    def method(self):
        print('P class method')
        super.method()
P1 = P()
P1.method()
```

* Polymorphism :- polymorphism is the concept of many
poly means many and morph means forms. polymorphism
means many forms
Ex :- Women :- is teacher, mother, housewife etc.

+ polymorphism are some methods :-

① Method overloading :-

* Method overloading is the compile time polymorphism
* Method overloading means having same name
method/fun in same/one class but different
arguments.

+ Method overloading is not required inheritance.

* write a program to show the method overloading to
find sum of two or three numbers.

```
→ class myclass:
    def sum(self, a=None, b=None, c=None):
        if (a != None and b != None and c != None):
            return (a+b+c)
        elif (a != None and b != None):
            return (a+b)
        else:
            return ("more than value required")
m1 = myclass()
print(m1.sum())
print(m1.sum(1))
print(m1.sum(2, 3.5))
print(m1.sum(1.2, 3.3, 1.2))
```


* Method overriding :-

- + method overriding is the runtime polymorphism.
- + method overriding means having same name but different class, arguments also same.
- + In method overriding inheritance is compulsory.

Ex:- class A:

```
def sayhi(self):
    print("I'm in A")
```

class B(A):

```
def sayhi(self):
    super().sayhi()
    print("I am in B")
```

~~main~~ bobj = B()

~~main~~ bobj.sayhi()

output:- I'm in A
I'm in B

* Operator overriding - or means giving Extended beyond there predefined operational meaning in python sum built-in operator or funⁿ has been represented as magic operators to show the different behavior for object or different classes this is called operator overloading.

- + write a program to overload binary operator + on object of class.

→ class A:

```
def __init__(self, A):
```

```
    self.a = A
```

```
def __add__(self, o):
```

```
    return self.a + o.a
```

```
obj1 = A(1)
```

```
obj2 = A(2)
```

```
obj3 = A("Greek")
```

```
obj4 = A('for')
```

```
print(obj1 + obj2)
```

```
print(obj3 + obj4)
```


* Overloading Comparison operator

class A :

```
def __init__(self, a)
```

```
    self.a = a
```

```
def __gt__(self, other):
```

```
    if (self.a > other.a):
```

```
        return True
```

```
    else:
```

```
        return False
```

```
obj1 = A(2)
```

```
obj2 = A(3)
```

```
if (obj1 > obj2):
```

```
    print('obj1 is greater than obj2')
```

```
else:
```

```
    print('obj2 is greater than obj1')
```

* Equality and less than operator :-

class A :

```
def __init__(self, a):
```

```
    self.a = a
```

```
def __lt__(self, other):
```

```
    if (self.a < other.a):
```

```
        return ('obj1 is less than obj2')
```

```
    else:
```

```
        return ('obj2 is less than obj1')
```

```
def __eq__(self, other):
```

```
    if (self.a == other.a):
```

```
        return ("Both are equal")
```

```
    else:
```

```
        return ("not equal")
```

```
obj1 = A(2)
```

```
obj2 = A(3)
```

```
print(obj1 < obj2)
```

```
obj3 = A(3)
```

```
obj4 = A(4)
```

```
print(obj3 == obj2)
```


* Ducks type philosophy :- Python has a type and the type is implicitly assign depending on the purpose for which the variable is used. same as if we want to call a method on an object we don't need to check the type of object and we don't need to check whether the method really belongs to that object or not. python follows ducks typing where the object is not check while accessing method on the object any object is accepted as long as the method is found in the object.

* write a program to check the object type whether the method exist in the object or not.

```

-> class india():
    def capital(self):
        print("new delhi is the capital of india")
    def language(self):
        print("Hindi is the primary language of india")
    def type(self):
        print("india is a developing contry")

class USA():
    def capital(self):
        print("washington is the capital of USA")
    def language(self):
        print("English is the primary language of USA")
    def type(self):
        print("USA is developing contry")

obj_ind = india()
obj_USA = USA()

for contry in (obj_ind, obj_USA):
    contry.capital()
    contry.language()
    contry.type()
    
```


OR without loop::
 def fun(obj):
 obj.capital()
 obj.language()
 obj.type()
 obj_india = India()
 obj_usa = USA()
 fun(obj_india)
 fun(obj_usa)

* class Dog:
 def bark(self):
 print('Bow, Bow')
 class duck:
 def talk(self):
 print('quack, quack')
 class human:
 def talk(self):
 print('Hello, Hi')
 def can_talk(obj):
 if hasattr(obj, talk):
 obj.talk()
 elif hasattr(obj, bark):
 obj.bark()
 else:
 print("wrong object passed")
 d = dog()
 can_talk(d)
 k = duck()
 can_talk(k)
 h = human()
 can_talk(h)

* Abstraction :- we will use the abstraction is hiding the data. we can show only necessary data

The abstract method has no body. Body is empty because this method implementing the another class. Ex. ATM machine

Ex. class Abstract class :
def abstractMethod (self):
pass.

* from abc import ABC, abstractMethod
class polygon (ABC):

Abstract method

def noofSides (self):

pass

class triangle (polygon):

overriding abstract method

def noofSides (self):

print ("I have 3 sides")

class pentagon (polygon):

override abstract method

def noofSides (self):

print ("I have 5 sides")

class hexagon (polygon):

override abstract method

def noofSides (self):

print ("I have 4 sides")

Driver code

R = triangle ()

R.noofSides ()

P = pentagon ()

P.noofSides ()

H = hexagon ()

H.noofSides ()


```
* class RBI:
    def interest (self):
        pass

class SBI (RBI):
    def interest (self):
        print ("SBI interest is 5%")

class HDFC (RBI):
    def interest (self):
        print ("HDFC interest is 7%")

s = SBI ()
s.interest ()
h = HDFC ()
h.interest ()

output: SBI interest is 5%
        HDFC interest is 7%
```

```
* class animal:
    def move (self):
        pass

class dog (Animal):
    def move (self):
        print ("I can bark")

class snake (Animal):
    def move (self):
        print ("I can hiss")

d = dog ()
s = snake ()
d.move ()
s.move ()
```


* Encapsulation :- The process of wrapping up variables and methods into a single entity is known as Encapsulation.

Class :-

Method	variable
--------	----------

Ex :- school bag

Ex class students:

```
def __init__(self, name, rank, points):
```

```
    self.name = name
```

```
    self.rank = rank
```

```
    self.points = points
```

```
def demofunc(self):
```

```
    print("I am", self.name)
```

```
    print("I got rank", self.rank)
```

```
st1 = students('steve', 1, 100)
```

```
st2 = students('charis', 2, 90)
```

```
st1.demofunc()
```

```
st2.demofunc()
```

* python access modifier :- Access modifier keyword that define the accessibility of a member, class or datatype in a program. This mainly used to restrict unwanted data manipulation by external program.

There are three access modifiers in python to understand the concept of Encapsulation and data hiding:

① public :- The public member is accessible from inside or outside the class.

② private Access modifier :- The private member is accessible only inside class. Define a private member by prefixing the member name with two underscores. Eg:- `__age`.

③ Protected Access modifier :- The protected member is accessible from inside the class and its sub-class. Define a protected member by ~~existing~~ prefixing the member name with underscore.

Ex :- `__points`.

* Exception Handling :-

The errors in the software are called as bugs and the process of removing them is called debugging.

* Error in the python :-

(1) Compile time error :- This are syntactical error found in the code due to which the program fails to compile. where python compiler display error message and the line number in case of compile time errors.

(2) Run time error :- when prgm can be executed the bytecode the flags run time error with. This are detected by the prgm only at run time.

Sometimes runtime error can not be eliminate in that case we should handle those errors using exception handling mechanism of python.

(3) logical errors :- This errors epic's follows in the logic of program. The programmer might be using a wrong formula or design of the program itself is wrong it is not detected by python compiler or prgm. The programmer is so responsible or dependent for them.

Compiler time error or logical error can be eliminated by the programmer by modifying the program source code in case of runtime error when the programmer notes which type of error occurs he has to handle them using exception handling. The runtime error which can be handle by the programmer are called exception.

* Exception :- Exception is a runtime error which can be handle by the programmer. If the programmer can go an error in the program we can do something to eliminate the harm cause by that error the it is called an exception.

Page No. _____
Date / /

All exceptions are represented as classes in python. The exceptions which are already available in python are called as built-in exceptions. The base class for all built-in exceptions is 'base exception class' from the exception class the subclasses standard error and warning are derived.

A programmer can also create its own exception called user-defined exception.

* Exception Handling :-

The purpose of exception handling is to make the program robust.

It will be required while designing such a program in any software requirement where the programmer should handle the error when an error can be handled they are called exceptions.

To handle exceptions in the program we should perform the following steps using some blocks:

* Exception Handling steps :-

- ① Try :- The try block lets you test a block of code for errors.
- ② Except :- The except block lets you handle the error.
- ③ else :- The else block lets you execute code when there is no error.
- ④ finally :- The finally block lets you execute code regardless of the result of the try and except blocks.

* Exception Handling :- When an error occurs or an exception as we call it, python will normally stop and generate an error message.

This exception can be handled using try statements.

Ex. The try block will be generate an exception, because x is not defined.

```
try:
    print(x)
except:
    print('An Exception occurred')
```

Note: Since the try block raise an error, the except block will be execute.

without the try block, the program will crash and raise an error.

Ex. Print one message if the try block raise a NameError and another for other occurs.

```
try:
    print(x)
except NameError:
    print("variable x is not defined")
except:
    print("something else went wrong")
```

* Else - you can used the else keyword to defined a block of code to be executed if no errors were raised.

Ex. In this example, the try block does not generate any error.

```
try:
    print('Hello')
except:
    print('something went wrong')
else:
    print('nothing went wrong')
```


Page No. _____
Date: / /

* Finally :- The finally block, if specified, will be executed regardless if the try block raise an error or not.

```
Ex. try :  
    print(x)  
except :  
    print('something went wrong')  
finally :  
    print("The try except is finished")
```

* This can be useful to close object and clean up resource

Ex. Try to open and write to a file that is not writable

```
try :  
    f = open("demoFile.txt")  
    try :  
        f.write("Hi exception")  
    except :  
        print('something went wrong when writing  
            to the file')  
    finally :  
        f.close()  
except :  
    print('something went wrong when opening the file')
```

* check age using exceptions

```
age = int(input('Enter your age:'))
```

```
try :  
    if (age < 18 or age > 55):  
        print('your Eligible')
```

```
    else :  
        print('your not Eligible')
```

```
except :
```

```
    print("something went wrong")
```


* Input any two numbers and checks if the number is divisible by 0 handle the exception

→ `n1 = int(input("Input 1st number"))`

`n2 = int(input("input 2nd number"))`

`try:`

`ng = n1 / n2`

`print('result = ' + ng)`

`except ZeroDivisionError:`

`print("Not divisible by zero")`

`finally:`

`print("Finally Block")`

* Userdefined Exception - This exception also known as custom exception.

The exception which are already available in python are called as built-in exception similar to this exceptions the programmer can also built his own exception which are introduced as user define exception

The raise statement is useful to raise an userdefine exception.

To create an userdefine exception as its own exceptions following steps has been follows.

- ① The programmer suppose to create its own exception class also should make a subclass to the built-in exception class
- ② When programmer suspicion the possibility of exception he should raise its own exception using raise statement
- ③ The programmer can insert the code inside the try blocks and catch the exception using except block.

* Write a program to create our own exception and display the message balance amount is less in the account of so and so person name. using bank data as person name and its account balance.

Page No. _____
 Date ____/____/____

```

→ class myException (Exception):
    def __init__ (self, arg):
        self.arg = arg
    def check (bk):
        print ("Name = {} balance = {}".format (k,v))
        for k,v in bk.items():
            if (v < 1000):
                raise myException.
bk = {'Laxmi': 1050, 'Raj': 4500, 'Rohit': 9350}
try:
    check (bk)
except myException:
    print ('Balance is less')
  
```

* Except principle amount period of time and rate of interest through the keyboard if the rate of interest is more than 100 the raise exception value out of range performed the task for multiple inputs.

```

→ def simple interest (amount, year, rate):
    try:
        if rate > 100:
            raise ValueError (rate)
        interest = (amount * year * rate) / 100
        print ('The simple interest is', interest)
        return interest.
    except ValueError:
        print ("interest rate is out of range" rate)
        exit ()
  
```

```

for i in range (3):
    print ('case', i+1)
    a = int (input ('principle Amount'))
    y = int (input ('period of time'))
    r = float (input ('Rate of interest'))
    simple interest (a, y, r)
    print (".....")
  
```


Page No. _____
Date: / /

```

* try :
  a = int(input("Enter value of a: "))
  b = int(input("Enter value of b: "))
  c = a/b
  print("The answer of a divisible by b: ", c)
except ZeroDivisionError as e:
    raise ValueError("Divisible failed")
    from e.
  
```

* Create an exception if the password is invalid.

```

→ try :
  a = int(input("What is your password"))
  if (len(a) != 8):
    raise
  print("Invalid password")
else:
  print("Successful login")
  
```

```

* class password (Exception):
    pass
  p = input("Enter the password")
  try :
    if (p != "xyz"):
      raise password
  except password:
    print("Invalid Error")
  
```

* Logging the Exception - The logging module is useful to store exception or error messages into a log file. It is useful to debug the program.

It is used to create the log file that can store all the error messages that may occur while executing a program.

There may be different level of error message which may represent numerical value.

Page No. _____
Date ____/____/____

This function logs a message with level error on this logger.

* following modules provide a set of function for simple login :-

① CRITICAL - 50

② ERROR - 40

③ WARNING - 30

④ INFO - 20

⑤ DEBUG - 10

⑥ NOTSET - 0

It is possible that we can set this default behavior as we needed.

To understand the different login messages we need to create a file for login message this is done by using basic configure of login module.

The method names can be critical, error, warning, info, debug, Notset etc.

We want to add message then we should used to method to set this error messages.

* write a program to store the message related by a exception into a log file

→ import logging
logging.basicConfig(filename=log.txt, level=logging.ERROR)

try :

a = int(input("Enter a no"))

b = int(input("Enter another no"))

c = a/b

except Exception as e :

logging.exception(e) :

else :

print("the result of division", c)

* write a program that create a logfile which error and critical message.

→ import logging

logging.basicConfig(filename=log.txt, level=logging.ERROR)


```
logging.error("there is mistake in program")  
logging.critical("Error in design of code")  
logging.info("single user can login")  
logging.warning("check plugin")  
logging.debug("line no 10 incorrect syntax")
```

* File handling :-

Every organisation depends on its data for continuing its business operation. If the data is lost, organisation it is to be close. There is the reason computers are primarily created for handling the data.

File handling is an important part of any web application.

Python has several functions for creating, reading, updating and deleting files. become easy using file handling.

* There are two types of python file :-

- ① Text file
- ② Binary file (text, image, Audio, video)

It is important to know how to create, store data in the file and retrieve the data from the files in python.

* Opening File :- we should use open function to open a file in the primary stage which is built-in function.

Syntax :-
`f = open("filename", "openmode", "buffering")`
`f = open("myfile.txt", "r")`

* There are different methods (modes) for file handling.

* 'w' = write the data into file, opens a file for writing, creates the file if it does not exist.

- * "r" = Read:- Default value. opens a file for reading, error if the file does not exists.
- * "a" = Append:- opens file for appending, creates the file if it does not exists.
- * "x" = Create:- creates the specified file, returns an error if the file exists.
- * "R+" = To Read and write, previous data in the file will not deleted
- * "W+" = To Read and write data.
- * "A+" = append and read the data of file, (create file does not exists)
- * wb, ab, atb, rtb, wtb are the modes for binary files.
- * Buffering is an option int use the of buffer for the file. Buffer represent a temporary block of memory.
- * closing File :- Ones the file is open but not close then the data of file may be corrupted or deleted. in some cases for that purpose close method is used.
- Syntax :- f.close()

* write a program to create a file to store the individual text in a file.

```
f = open("filenew.txt", "w")
p = input("Enter text :")
f.write(p)
f.close()
```

cheds writing data to Read

```
f.open("filenew.txt", 'r')
print(f.read())
```


* write a program to store a group of string into a .txt file and display the content on separate line as per the user choice.

```
→ f = open('myfile.txt', 'w')
print("Enter text end line @")
while (str != "@") :
    str = input()
    if (str != "a") :
        f.write(str + "\n")
f.close()
```

```
→ f = open("myfile.txt", 'r')
print("using read () all data")
print(f.read())
print("only 5 characters")
print(f.read(5))
print(f.readline())
print(f.readlines())
```

* Delete the file - To delete a file import os module and run os.remove method.

```
Ex. import os
os.remove("myfile.txt")
```

* To remove directory and folder :-

```
import os
os.rmdir("untitled folder")
```

* write a program to delete a textfile if it is exist either display the message file does not exists.

```
→ import os
if os.path.exists("myfile3.txt") :
    os.remove("myfile3.txt")
else:
    print("file doesn't exists")
```


Page No. _____
Date ____/____/____

* write a program to input text form of data and count number of lines, word and character in the existing text file.

```
→ import os, sys
fname = input("Enter Filename")
if os.path.isfile(fname):
    f = open(fname, 'r')
    print("open successful")
else:
    print(fname + "doesnot exist")
    sys.exit()
cl = cw = cc = 0
for line in f:
    words = line.split()
    cl += 1
    cw += len(words)
    cc += len(line)
print("No. of lines = ", cl)
print("No. of words = ", cw)
print("No. of characters = ", cc)
```

* Seek and Tell method :-

In python seek() function is used to change the position of the file handle to a given specific position

Syntax :- seek(offset, fromhere)
seek(5, 20)

* offset represent how many bytes to move

fromhere represent from which position to move

We can also used seek method as a single parameter which changes current file position to the given location and return rest of the line.

```
f = open("demo.txt", "r")
f.seek(4)
print(f.read())
```


* write a program to demonstrate seek method using file handling.

```
→ f = open('HEH.txt', 'r')
f.seek(20)
print(f.tell())
print(f.readline())
f.close()
```

```
* f = open("data.txt", "rb")
f.seek(-10, 2)
print(f.tell())
print(f.readline().decode('utf-8'))
f.close()
```

* Tell method :- The tell() method returns the current file position in a file stream.

* Create a class employee having members id name, salary as details as instance variable.

```
→ class employee:
    def __init__(self, id, salary, name):
        self.id = id
        self.salary = salary
        self.name = name
    def show(self):
        print("employee id", self.id)
        print("employee salary", self.salary)
        print("employee name", self.name)
```

* write a program to pickle employee class object.

```
import emp, pickle
f = open('emp.dat', 'wb')
n = int(input("How many employees"))
for i in range(n):
```


Page No. _____
Date: ____/____/____

```

n = input ("Emp name")
id = int (input ("Emp id"))
sal = int (input ("Emp salary"))
e = emp (n, id, sal)
pickle.dump (e, f)
f.close ()

```

* write a program to unpickle or object deserialisation using emp class object

```

→ import emp, pickle
f = open ('emp.dat', 'rb')
while True:
    try:
        obj = pickle.load (f)
        obj.show ()
    except EOFError:
        print ("Employee File")
        break
f.close ()

```

* load method reads and object from binary file and written to into an object. unpickling is done using load method is also known deserialisation unpickle is a process where by a bytes string is converted into a class object it mean reading a class object from the file.

* dump method :- is used for serialisation or pickeling when object should be stor into a binary file in the form of bytes pickle is the process of converting a class object into a bytes stream so that it can be stor into a file.

* With statement :-

With statement can be use while opening a file the advantages of with statement is that it will take care of closing a file which is open by it.

We need not closed the file explicitly with statement will closed the file before the exception is handle.

if the file is again open with statement take care to closed it priority basis.

* Write a program to represent use of with

① without using with statement :-

```
file = open("file path", "w")
```

```
file.write("hello world!")
```

```
file.close()
```

② without using with statement :-

```
file = open("file path", 'w')
```

```
try:
```

```
    file.write("hello world")
```

```
finally
```

```
    f.close()
```

③ Using with statement :-

```
with open('file-path', 'w') as file:
```

```
    file.write('hello world!')
```

* Ziping and Unziping File :-

python provide ziping and unziping of file data in ziping the file contains following two things.

- ① Contains are compress hence the file will be reduce.
- ② format of the data will be change making it unreadable.

Page No. _____
Date ____/____/____

* write a program to represent compress the content of the file.

```
→ from zipfile import *  
f = ZipFile("test.zip", 'w', ZIP_DEFLATED)  
f.write("file1.txt")  
f.write("file2.txt")  
f.write("file3.txt")  
f.close()
```

* To unzip the contains of compress file and get back the original contain we can use ~~zipfile~~ ZipFile class object in the read mode to extract all the file zip object extract all method is used.

```
→ F = ZipFile("test.zip", 'r')  
f.extractall()  
f.close()
```

* write a program to work on zipfile using printing all the contains of it.

```
→ from zipfile import ZipFile  
filename = "my-python-files.zip"  
with ZipFile(filename, 'r') as zip:  
    zip.printdir()  
    print("Extracting all the files now ---")  
    zip.extractall()  
    print("Done")
```

Print dir method is used to print table of contents from archive and extract all method of the zip file to the current working directory we can also called extract to extract any file by specifying the path in the zip file.

* Working with Dictionary 8 - os module

os module represents operating system dependent functionality it is useful to perform some simple operation on the directories like finding currently working directory, changing to a particular directory, renaming or deleting the directory and listing the content of directory.

The system of os module is useful to run commands all the executable programs from our python file.

To get current working directory in os module getcwd method is used.

* import os module.

```
import os
```

```
# Get the current working
```

```
# directory (cwd)
```

```
cwd = os.getcwd()
```

```
print("current working directory:", cwd)
```

* write a program to create subdirectory and then subdirectory and current directory.

```
os.mkdir("testdir")
```

```
# os.mkdir("testdir/check")
```

* write a program to change the current directory.

```
print("current working directory:" (cwd))
```

```
goto = os.chdir("F:/practice")
```

```
d = os.getcwd()
```

```
print("current working directory:" d)
```


* multithreading :-

A thread represent separate path of execution of group of statement. It can be imagin as a light weight process that executed a group of statement.

The statement of every python program are executed by default thread called main thread.

A thread represent execution of statement the way statement are executed are of two type.

(1) single tasking

(2) multitasking.

(1) single tasking :- only one task in the given to the processor at a time these means we resting lot of processor time and has to set of idle without from long time this is a single drawback in single tasking.

(2) Multitasking :- i.e. multiple task has been performed by the processor simultaneously this called multitasking. The main advantages multitasking is to use the processor time in better way. i.e. processor not seeking idle. so we can complete several task at a time and those achieve good performance.

* multitasking is a two types :-

(1) process base tasking

(2) thread base tasking.

In python it is possible to create multiple threads and execute them different parts of the program simultaneously it is called as concurrent programming.

When more than one thread at running at a time the data of one thread available to another thread in such cases a possibility that data may undergoes unwanted manipulation these will lead wrong result it means that pvm is not threadsafe hence pvm uses

(GIL) Global Interpreter Lock that allows only single thread to executed at any given movement. GIL does not allow more than one thread to run at a time.

* Uses of Thread :-

- ① mainly used in several site program.
- ② Also use in game and animation.

* Creating threads in python :-

- There are different ways of creating thread in python
- (i) creating a thread without using thread class.
 - (ii) Creating a thread by creating a subclass to thread class.
 - (iii) Creating a thread without creating subclass to thread class.

* Creating a thread without using thread class.

We can create a thread by creating object of thread class and pass the function name as target.
Syntax :- Thread (target = function, args = (param, arg, ...))

Ex: `import threading`

`def print_hello(n):`

`print ("Hello, how old are you", n)`

`print ("Hello, how old are you", n)`

`t1 = threading.Thread (target = print_hello, arg = (18,))`

`t2 = threading.Thread (target = print_hello, arg = (28,))`

`t1.start ()`

`t2.start ()`

* Thread class method :-

① start :- It is used to initialize thread activity
 i.e (Thread get boar)

② run :- it is used to define actual thread activity.

② join :- it is used to block execution of another code until the thread terminate.

→ import threading

→ def print_hello(n)

print("Hello, how old are you", n)

t₁ = threading.Thread(target = print_hello, args = (20,))

t₂ = threading.Thread(target = print_hello, args = (28,))

t₁.start()

t₁.join()

t₂.start()

print("Thank you!")

* creating a thread by creating a subclass to subclass.

from threading import Thread

class myThread(Thread):

def run(self):

for i in range(1, 6):

print(i)

t₁ = myThread()

t₁.start()

t₁.join()

print("Thread execution complete")

* create a thread to access the instance of variable by passing the name of person.

→ from threading import Thread

class person(Thread):

def __init__(self, nm)

self.nm = nm

def run(self):

print("who are you?", self.nm)

print("Hi, welcome to python", format(self.nm))

p₁ = person("Rohit")

p₂ = person("Ashwin")

p₁.start()

p₁.join()

p₂.start()

* Creating a thread without creating a subclass to Thread class.

We can create an independent class that does not inherit from Thread class then the next step is to create thread by creating an object to thread class where target represents function or method.

* write a program to create a thread that act on object of a class that is not derived from the thread class.

→ From threading import *

class mythread:

def display (self):

i = 0

print (current_thread().getname())

while (i <= 10):

print (i)

i = i + 1

obj = mythread()

t₁ = Thread (target = obj.display)

t₂ = Thread (target = obj.display)

t₃ = Thread (target = obj.display)

t₁.start()

t₁.join()

t₂.start()

t₂.join()

t₃.start()

* multitasking using multiple thread :-

write a program that performed tasks using two threads simultaneously where creator is the class where two different thread name has been created.

→ From threading import *

class threader:

def __init__ (self, str):

Page No. 0
 Date / /

```

self.str = str
def movieshow(self):
    for i in range(1,6):
        print(i, " : ", self.str)
        sleep(0.2)
t1 = threading.Thread(target=self.movieshow)
t2 = threading.Thread(target=self.movieshow)
obj1 = Thread(target=t1.movieshow)
obj2 = Thread(target=t2.movieshow)
obj1.start()
obj2.start()
    
```

* Create a class railway where reserve method represents wanted number of seats and represents available seats either some no seats are available. Create two thread acting on the same method to allot seats for the passenger.

```

from threading import *
from time import *

class railway:
    def __init__(self, available):
        self.available = available
    def reserve(self, wanted):
        print("available seats are", self.available)
        if (self.available >= wanted):
            name = current_thread().getName()
            print("%d berths wanted to %s" % (wanted, name))
            sleep(1)
            self.available -= wanted
            print(wanted, " berths are reserved")
        else:
            print("No berths are allotted")

obj = railway(1)
t1 = Thread(target=obj.reserve, args=(1,))
t2 = Thread(target=obj.reserve, args=(1,))
t1.start()
t2.start()
    
```


* Thread synchronization :-

When a thread is already acting on an object preventing any other thread from acting on some object. it called thread synchronisation or thread safe.

The object on which threads are synchronize are called synchronize object or mutex.

* Synchronisation is done by two ways :-

- (1) Using locks
- (2) Using semaphores.

(1) Locks :- locks can be used to lock the object on which thread is acting when a thread enters the object it locks and after the execution is completed it will unlocks the object and comes out of it. it is like a room with only one door. When we create a lock by creating an object of locks class as follow.

- (1) create an object `l = locks()`
- (2) TO lock the current object use should used `acquire` method `= l.acquire()` to unlocks or release the object we can used `release` method.

* write a program to achieve thread synchronization using lock
→ from threading import *

class railway :

```
def __init__(self, available):
    self.available = available
    self.l = locks()
```

```
def reserve(self, wanted)
```

```
    self.l.acquire() # acquire
```

```
    print("%d berths are available" % self.available)
    if (self.available > wanted):
```


Page No. _____
Date ____/____/____

```

print("%s wanted on of berth %d" % (current
thread().getname(), wanted))
print("%d berths reserved successfully, %d wanted"
self.l.release())
else:
    print("No berths are available")

```

```

r = railways(3)
t1 = thread(target = r.reserve, arg (1,))
t2 = thread(target = r.reserve, arg (3,))
t1.start()
t2.start()

```

* Semaphores :- is an object that provide synchronisation base on counter. a semaphore is created of an object of class. if the counter value is not given the default value of counter will be one when acquire method is call the counter gets decremented by one and when release method is called it is incremented by one these methods are used as follows.

```

l = semaphore (counter value)
l.acquire()
l.release.

```

* Deadlocks of Thread :- If we synchronise the thread there is possibility of other problems like deadlocks since two threads of acting on similarly the result may not be reliable hence case have to lock this two object for this purpose we may used the method in different order where may be program can be used..

* Write a program to create a method books ticket and cancel ticket performed synchronisation using locks.


```

from threading import *
def bookticket (self):
    l1 = acquire ()
    print ("Ticket get booked")
    l2 = acquire ()
    print ("Ticket Booking Complete")
    l1 = release ()
    l2 = release ()
def cancelled (self):
    l2 = acquire ()
    print ("Ticket get canceled")
    l1 = release ()
    l2 = release ()
t1 = Thread (target = bookticket)
t2 = Thread (target = cancel)
t1.start ()
t2.start ()

```

Note:- All a dead lock means is that one or more thread are blocks by making any progress so you can simulate it with a single thread.

* Communication between threads :-
like consumer thread is waiting for a producer to produce the data in a python program were producer or consumer thread with each other through boolean type variable.

* Write a program where producer or consumer threads communicate each other through given variables.
→ from threading import time.

class producer :

```

def __init__(self):
    self.lst = []
    self.dataproducer = False.

```


Page No. _____
Date: ____/____/____

```

def producer (self):
    for i in range (1, "):
        self.lst.append (i)
        sleep (i)
        print ("Item produced")

class Consumer:
    def __init__ (self):
        self.prod.prod

    def consume (self):
        while self.prod.dataprodover == False:
            sleep (0.1)
            print (self.prod.lst)

```

p = producer ()

c = Consumer ()

t₁ = Thread (target = p.produce)

t₂ = Thread (target = c.consume)

t₁.start ()

t₂.start ()

* To improve the efficiency of communication of betⁿ the thread python provides two important ways which are as below.

① Using wait () and notify ()

② Using queue.

using notify and wait method condition class of threading module is useful to improve speed of the communication between the threads the condition class object is called condition variable.

cr = Condition ()

modify method is useful to immigiated informed the existing thread that the data production has been complete. notify method is useful to informed single thread and notify all method is useful to informed all thread are waiting.

wait method is used to accept the time and.

Page No. _____
Date: / /

terminates when notify or notify all () method executed. wait method will not waste single milli second to receive the data after execution this will the wait till notification is receive.

* write a program to represent thread communication is done through notify and wait method of condition object.

→ from threading import *
from time import *

class produce :

def __init__(self):

self.lst = []

self.cv = condition()

def produce(self):

self.cv.acquire()

for i in range (1,11):

self.lst.append(i)

sleep(1)

print("Item produced")

self.cv.notify()

self.cv.release()

class consumer :

def __init__(self):

self.prod = prod

def consumer(self):

self.prod.cv.acquire()

self.prod.cv.wait(timeout=0)

self.prod.cv.release()

print(self.prod.lst)

p = produce()

c = consumer()

t1 = Thread(target=p.produce)

t2 = Thread(target=c.consumer)

t1.start()

t2.start()


```

import threading
import time
import random
class subclass:
    def __init__(self):
        self.x = []
    def produce_item(self, x-item):
        print("producer adding on item to the list")
        self.x.append(x-item)
    def consume_item(self):
        print("consuming from the list")
        consumed_item = self.x[0]
        print("consumed item:", consumed_item)
        self.x.remove(consumed_item)
    def producer(subclass_obj, condition_obj):
        r = random.randint(1, 3)
        print("random number selected was:", r)
        for i in range(1, r):
            print("producing an item, time it will take",
                  (second):", str(i))
            print("producer acquiring the lock: condition_obj",
                  .acquire())
            try:
                subclass_obj.produce_item(i)
                condition_obj.notify()
            finally:
                condition_obj.release()
    def consumer(subclass_obj, condition_obj):
        condition_obj.acquire()
        while True:
            try:
                subclass_obj.consume_item()
            except:
                print("no item to consume, list empty")
                print("waiting for 10 second")

```



```

value = condition_obj.wait(10)
if value:
    print("item producer notified")
    continue
else:
    print("waiting timeout")
    break
condition_obj.release()
if __name__ == "__main__":
    condition_obj = Condition()
    subclass_obj = Subclass()
    pro = Threading.Thread(target=producer, args=(subclass_obj, condition_obj))
    pro.start()
    con = Threading.Thread(target=consumer, args=(subclass_obj, condition_obj))
    con.start()
    pro.join()
    con.join()
    print("producer consumer code executed")

```

* GUI (Graphical User Interface) :-

- GUI :- Graphical User Interface.
- Tkinter :- is the GUI library for the python. It is fastest and easiest way to create a GUI application in python and its most commonly used for GUI application.

* Steps to create GUI Application :-

- ① import the module - tkinter
- ② create the main window
- ③ Add any number of widgets to the main window
- ④ Apply the event trigger on the widgets.

* Import and methods of Tkinter

- first import the module tkinter.

import tkinter

- There are two main method.

① Tk()

② mainloop()

* Methods of Tkinter :-

* Tk() :- To create an main loop window tkinter provide a method.

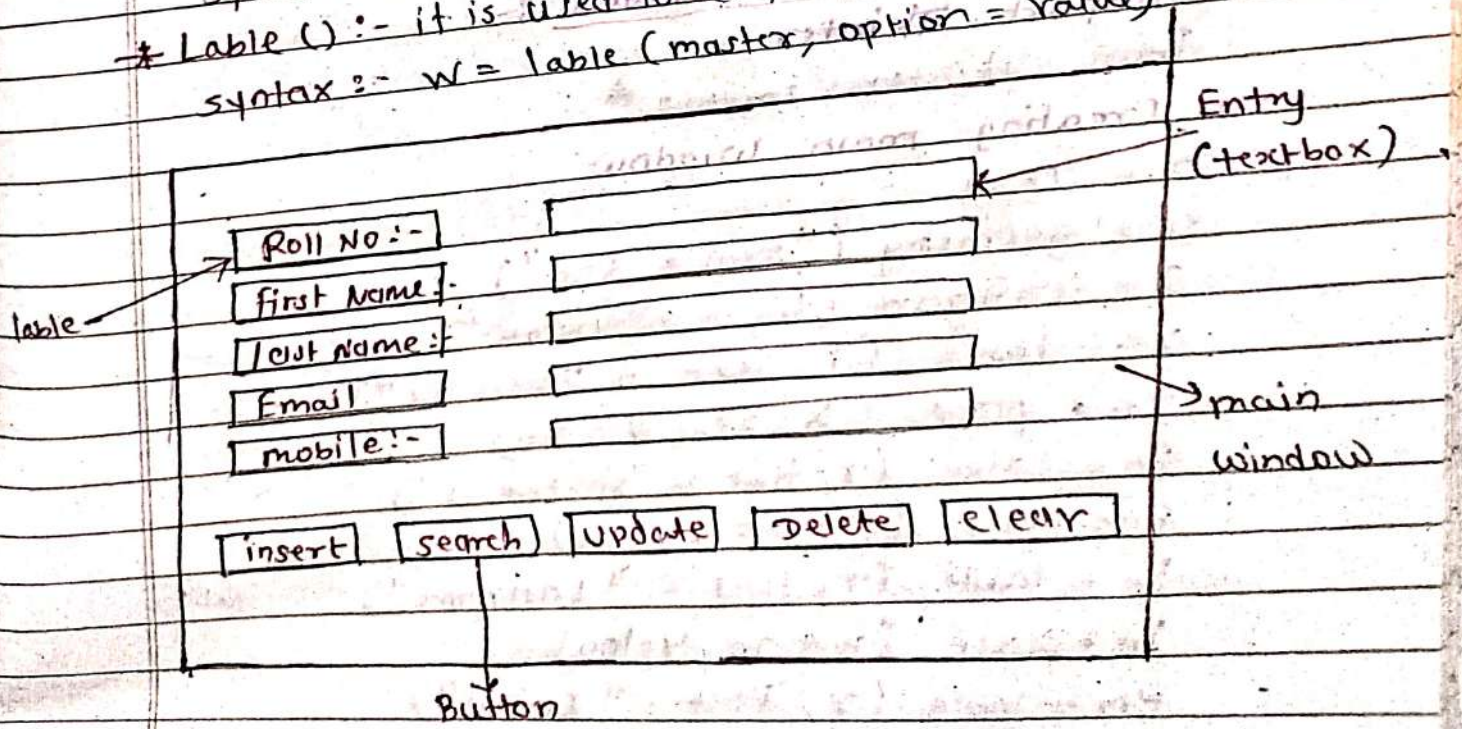
* Mainloop() :- There is method know by the name mainloop(), it is used to run your application.

* Widgets of Tkinter :-

* Button() :- To add a button in your application
syntax :- W = Button(master, option = value)

* Entry() :- it is used to take input from user.
syntax :- W = Entry(master, option = value)

* Label() :- it is used to display a text in your application
syntax :- W = Label(master, option = value)



* GUI program :-

* # main window creating

import Tkinter

from tkinter import *

Creating main window

r = Tk()

mainloop method called

mainloop()

* Creating customize window :-

import Tkinter

from Tkinter import *

Creating main window

r = Tk()

r.geometry("500x500")

r.title("First GUI")

r.configure(bg="yellow")

mainloop()

* Adding widgets into window :-

→ # import Tkinter

from Tkinter import *

Creating main window

r = Tk()

r = geometry("500x500")

r = configure(bg="yellow")

rn = Label(r, text="Roll No")

rn.place(x=20, y=20)

fn = Label(r, text="Firstname")

fn.place(x=20, y=60)

ln = Label(r, text="Lastname")

ln.place(x=20, y=100)

em = Label(r, text="Email")

em.place(x=20, y=140)

Adding Entry Box into main window :-

```
ern = Entry()
```

```
ern.place(x=100, y=20)
```

```
ern = Entry()
```

```
ern.place(x=100, y=60)
```

```
ern = Entry()
```

```
ern.place(x=100, y=100)
```

```
ern = Entry()
```

```
ern.place(x=100, y=140)
```

Adding Buttons into main window :-

```
button1 = Button(r, text="Button1", bg="white")
```

```
button1.place(x=20, y=200)
```

```
mainloop()
```

* Database :-

* Database Management System (DBMS) :-

A database management system (DBMS) is system software for creating and managing database. A

DBMS makes it possible for end users to create read, update, delete and protect data in a database.

* Database :-

* The python standard for database interfaces is the python DB-API most python database interfaces adhere to this standard.

* you can choose any database your application

ex:- Gladfly, mysql, msql, postgresql, microsoft sql server 2000, informix, interface, oracle, sybase.

* Database Installation Step 8-9

* First install pymysql library using following cmd command.

`Pip install pymysql`

* Install SQLyog software for database work.

* Step for working with Database 8-

- Create database in sqlyog / mysql
- import the API module.
- Acquiring a connection with the database
- Issuing sql statement
- closing the connection.

* Import the API module :-

`import pymysql`

* Acquiring a connection with the database :-

`conn = pymysql.connect (Host = " ", Database = " ",
User = " ", password = " ", port = 3306)`

* Issuing SQL statement :-

`cursor = conn.cursor()`

`query = " "`

`cursor.execute(query)`

* Closing the connection :-

`conn.commit()`

`conn.close()`

* Program Example :-

→ `# import Database API`

`import pymysql`

`# Database Connection function.`

`def createConn():`

`return pymysql.connect (host = "localhost", database =
"learnvnm", user = "root", password = " ", port = 3306)`

Table created function

```
def createTable():
```

```
    conn = createConn()
```

```
    cursor = conn.cursor() # helping to execute your query.
```

```
    query = "create table student (sid int primary key  
        auto-increment, name varchar(50), email varchar(50),  
        city varchar(50))"
```

```
    cursor.execute(query)
```

```
    conn.commit()
```

```
    conn.close() ← print("Table created")
```

createTable() # calling create table function

* Insert record in database :-

```
def InsertData(name, email, city):
```

```
    conn = createConn()
```

```
    cursor = conn.cursor()
```

```
    args = (name, email, city)
```

```
    query = "insert into student (name, email, city)  
        values(%.s, %.s, %.s)"
```

```
    cursor.execute(query, args)
```

```
    conn.commit()
```

```
    conn.close()
```

```
    print("Data inserted")
```

```
    conn.close()
```

```
n = input("Enter your name :")
```

```
e = input("Enter your email :")
```

```
c = input("Enter your city :")
```

```
InsertData(n, e, c)
```

* Fetch All data from database :-

database connection function steps. ---

```
def showAllData():
```

```
    conn = createConn()
```

```
    cursor = conn.cursor()
```

```
    query = "select * from student"
```

```
    result = cursor.fetchall() ← cursor.execute(query)
```



```
for i in result :
```

```
    print (i)
```

```
showAllData ()
```

* Fetch Data by id from Database :-

```
def ShowAllDataById ( sid ) :
```

```
    Conn = create.conn ()
```

```
    Cursor = Conn.cursor ()
```

```
    args = (sid)
```

```
    query = "select * from student where sid = '%s'"
```

```
    Cursor.execute (query, args)
```

```
    result = Cursor.fetchall ()
```

```
    for i in result :
```

```
        print (i)
```

```
showAllData ()
```

```
sid = int (input ("Enter your id :"))
```

```
ShowAllDataById (sid)
```

* Update data and delete data from database :-

Database connection step ----- port=3306

* update data into database :-

```
def update_data (name, email, city, sid) :
```

```
    Conn = createConn ()
```

```
    Cursor = Conn.cursor ()
```

```
    args = (name, email, city, sid)
```

```
    query = "update student set name = '%s', email = '%s',
```

```
city = '%s' where sid = '%s'"
```

```
    Cursor.execute (query, args)
```

```
    Conn.commit ()
```

```
    print ("Data updated")
```

```
    Conn.close ()
```

```
showAllData ()
```

```
sid = int (input ("Enter yid :"))
```

```
n = input ("Enter y name :")
```

```
e = input ("Enter y Email :")
```

```
c = input ("Enter y city :")
```


updateData (n, e, c, sid) # for updating
showAllData () # for showing update data.

* Delete data from Database :-

```
def Deletedata (sid):
    conn = createConn ()
    cursor = conn.cursor ()
    args = (sid)
    query = "delete from student where sid = '%s'"
    cursor.execute (query, args)
    conn.commit ()
    print ("Data Deleted")
    conn.close ()

showAllData ()
sid = int (input ("Enter your id = "))
# calling delete method
Deletedata (sid)
showAllData ()
```

* Database Steps :-

① Go To mysql and create database or database name.

import pymysql

def createConn () :

return pymysql.connect (host = "localhost", database = "pramod",
user = "root", password = "pramod123", port = 3306)

def CreateTable () :

conn = createConn ()

cursor = conn.cursor ()

query = "createtable student (id int primary key auto-
increment, name VARCHAR (50), email VARCHAR (50),
city VARCHAR (50))

cursor.execute (query)

conn.commit ()

print ("Table created")

conn.close()

createTable()

def InsertData (name, email, city):

conn = createconn()

cursor = conn.cursor()

args = (name, email, city)

query = "insert into student (name, email, city) values ('.s', '.s', '.s')"

cursor.execute(query, args)

cursor.commit()

print("Data inserted")

conn.close()

n = input("Enter your name = ")

e = input("Enter your email = ")

c = input("Enter your city = ")

InsertData(n, e, c)

def showAllData():

conn = createconn()

cursor = conn.cursor()

query = "select * from student" ~~where id = 4~~

cursor.execute(query)

result = cursor.fetchall()

for i in result:

print(i)

showAllData()

id = int(input("Enter your id No: "))

showAllDataById

def showAllDataById(id):

conn = createconn()

cursor = conn.cursor()

args = (id)

query = "select * from student where id = .s"

cursor.execute(query, args)

result = cursor.fetchall()

for i in result:


```

print (i)
showAllData ()
id = int (input ("Enter your id No:"))
showAllDataByID (id)
def updateData (name, email, city, id):
    conn = createConn ()
    cursor = conn.cursor ()
    args = (name, email, city, id)
    query = "update student set name = '%s',
            email = '%s', city = '%s' where id = '%s'"
    cursor.execute (query, args)
    conn.commit ()
    print ("Data Updated")
    conn.close ()

```

```

showAllData ()
id = int (input ("Enter your id:"))
n = input ("Enter your name:")
e = input ("Enter your email:")
c = input ("Enter your city:")
updateData (n, e, c, id)
showAllData ()

```

```

def deleteData (id):
    conn = createConn ()
    cursor = conn.cursor ()
    args = (id)
    query = "delete from student where id = '%s'"
    cursor.execute (query, args)
    conn.commit ()
    print ("Data Deleted")
    conn.close ()

```

```

showAllData ()
id = int (input ("Enter your id:"))
deleteData (id)
showAllData ()

```